Smart Delivery and Route Optimization Engine

Assignment 5 — Strategy Pattern

Student: Nurakhmet Mukhtiar
Group: SE-2517
Technology: Java 17+

This report describes a route optimization engine that selects delivery routes using interchangeable algorithms. It demonstrates the Strategy Pattern, hard business constraints, automatic selection, runtime switching, explanation, testing, and an extensible fifth strategy.

1. Problem and architecture

A logistics system can favor delivery speed, price, environmental impact, or a weighted tradeoff. The Strategy Pattern encapsulates each route ranking algorithm behind RouteStrategy. RoutePlanner is the Context: it stores a strategy abstraction and delegates planning without implementing any of the algorithms.

Context: RoutePlanner. **Strategy interface:** RouteStrategy. **Concrete strategies:** FastestRouteStrategy, CheapestRouteStrategy, EcoFriendlyRouteStrategy, BalancedRouteStrategy, and SafestRouteStrategy. **Automatic selection:** StrategySelector maps request conditions to strategy factories. **Result:** RouteDecision contains the selected route, an explanation, and rejected route reasons.

UML class diagram

```
RoutePlanner --delegates--> RouteStrategy [interface]
                                    ^
                  AbstractRouteStrategy
        ^       ^       ^       ^       ^
      Fastest Cheapest Eco   Balanced Safest

StrategySelector --creates--> RouteStrategy
RouteStrategy --reads--> Route + DeliveryRequest
RouteStrategy --returns--> RouteDecision
Route --origin/destination--> Location
```

RoutePlanner aggregates RouteStrategy. AbstractRouteStrategy implements the common input validation and hard constraints. Each concrete strategy extends it. StrategySelector creates a strategy from a DeliveryRequest, while each strategy evaluates Route candidates and returns a RouteDecision.

UML sequence diagram

```
Client -> StrategySelector: select(urgent request)
StrategySelector -> Client: FastestRouteStrategy
Client -> RoutePlanner: setStrategy(fastest)
Client -> RoutePlanner: plan(routes, request)
RoutePlanner -> FastestRouteStrategy: selectRoute(routes, request)
FastestRouteStrategy: filter constraints; rank time + traffic
FastestRouteStrategy -> RoutePlanner: RouteDecision with reasons
RoutePlanner -> Client: RouteDecision
```

2. Domain and hard constraints

The domain includes Route, Location, DeliveryRequest, and DeliveryPriority. Eight candidate routes appear in the demonstration. Every algorithm filters constraints before scoring: route availability, hazardous cargo permission, maximum budget, deadline, and maximum emissions. No candidates or no eligible route causes an explicit NoSuitableRouteException.

3. Route strategies and formulas

Fastest: Minimize estimatedMinutes + trafficDelayMinutes; traffic is part of the ranking.

Cheapest: Minimize baseCost + distanceKm × costPerKm + priority surcharge. Surcharge is +1500 urgent, 0 standard, and –200 economy.

Eco friendly: Minimize 0.8 × carbonEmissionKg + 0.2 × distanceKm. This formula belongs in its Strategy because it is one interchangeable ranking behavior.

Balanced: Minimize weighted normalized time, cost, distance, and emissions: 0.40, 0.30, 0.10, and 0.20. Each dimension is normalized across eligible routes to [0, 1]; weights must be finite, non-negative, and sum to 1.

Safest (fifth strategy): Minimize 0.6 × effectiveMinutes + 10 × carbonEmissionKg, an explicit educational proxy. It is added without changes to RoutePlanner.

StrategySelector decides which algorithm should be used; the selected Strategy decides how to rank routes. Its mapping uses registered factories. Default rules select Fastest for urgent, Cheapest for economy, Eco for an eco-preferred standard request, and Balanced for other standard requests.

4. Runtime switching and comparison output

DeliveryApplication compares Fastest, Cheapest, Eco, and Balanced on the same request and uses one RoutePlanner instance while switching it among Fastest, Eco, and Cheapest. Its output for request #105 is:

```
Fastest: R2 | 22 min | 1760.00 | estimated 20 min + traffic 2 min
Cheapest: R3 | 38 min | 1500.00
Eco friendly: R3 | 38 min | 1500.00 | eco score 2.960
Balanced: R6 | 28 min | 1505.00 | normalized score 0.256
Runtime changes: Fastest → R2; Eco → R3; Cheapest → R3
Automatic urgent selection: Fastest
```

5. Strategy Pattern, Bridge, and SOLID

Strategy swaps one behavior at runtime behind a common contract. Bridge separates two independently varying dimensions, commonly an abstraction hierarchy and an implementation hierarchy. Here, the central relationship is one Context delegating to an interchangeable route-selection algorithm.

SRP: planning, algorithm selection, route ranking, and domain data have separate responsibilities. **OCP:** SafestRouteStrategy is an added algorithm that does not modify RoutePlanner. **DIP:** RoutePlanner depends on RouteStrategy rather than concrete strategy classes.

6. Tests, Git history, and limitation

The JUnit 5 suite contains 23 meaningful tests spanning the required strategies, invalid balanced weights, runtime switching, automatic selection, availability and cargo restrictions, budget and deadline limits, no route and empty input, invalid domain values, result explanation, and strategy comparison. Executed locally using JUnit 5.11.3 and JUnit Platform 1.11.3: 23 tests passed, 0 failed. Maven was not installed; on a Maven-equipped system run `mvn test`. Surefire writes reports to `target/surefire-reports/`.

The Git history has 11 meaningful commits. SafestRouteStrategy has its own commit. The limitation is illustrative in-memory route data with fixed coefficients: production deployment needs live traffic, measured emission and cost data, and stakeholder-validated weights. The safety score is not a certified safety model.